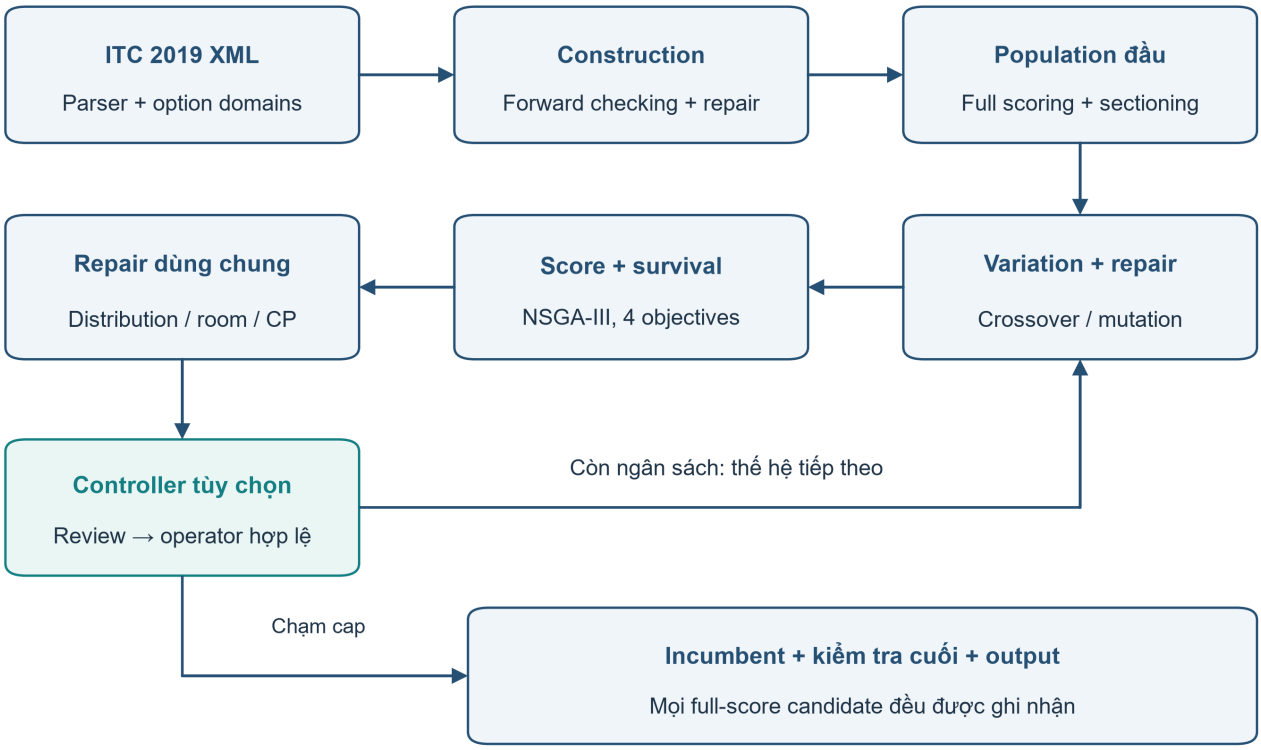
BÁO CÁO 01 / KIẾN TRÚC VÀ VẬN HÀNH

Solver hiện tại

Mô tả implementation đang có, không phải thiết kế dự kiến hay báo cáo benchmark.

Kết luận chính

Ba pipeline dùng cùng một NSGA-III có repair. LLM chỉ bổ sung quyết định WHERE, HOW và WHEN; không thay scorer, representation hay cơ chế chọn lọc của solver.



Hình 1. Luồng rút gọn của solver. Full scoring cập nhật incumbent trước survival; cap bao gồm generation, logical evaluations và thời gian. Controller được bỏ qua ở baseline.

- **Baseline:** NSGA-III + construction/repair/monitor chung; không gọi API.
- **Single-agent:** một agent chọn vùng, operator/budget và lần review tiếp theo.
- **Inspector-Planner:** Inspector chọn WHERE; Planner chọn HOW và WHEN sau khi target được xác minh.
- **Phạm vi báo cáo:** dữ liệu và scorer; search/repair; controller; ngân sách; output; trạng thái và giới hạn.

Nguồn chính: PIPELINES.md; IMPROVEMENT_HISTORY.md (H01-H30); profiles/pilot.json. Giữ nguyên các file nguồn và logic solver trong lần biên tập này.

01 / DỮ LIỆU VÀ ĐÁNH GIÁ

Một chromosome biểu diễn điều gì?

- **Đơn vị dữ liệu:** mỗi XML ITC 2019 là một instance độc lập. Pilot/subset chọn các XML nguyên vẹn, không cắt tùy ý class hoặc ràng buộc trong một instance.
- **Chromosome dài 2n:** n chỉ số time-option rồi n chỉ số room-option. Class không cần phòng dùng room index -1. Gene chọn trong domain cho sẵn của từng class.
- **Student sectioning không nằm trong gene:** scorer giải mã việc phân sinh viên theo course/config/subpart, parent relationships và class limits.

Hard và soft được tách riêng

Thành phần	Ý nghĩa
room_conflicts	Các cặp class trùng phòng và trùng lịch.
room_unavailable	Placement giao với khoảng phòng không khả dụng.
hard_distributions	Vi phạm các distribution constraints bắt buộc.
unassigned_requests	Yêu cầu đăng ký course chưa được gán đủ.

- **Hard** bằng tổng bốn thành phần trên. Hard 0 chỉ là khả thi theo implementation nội bộ.
- **Bốn objective:** weighted time, room, soft distribution và student conflict. NSGA-III survival vẫn đa mục tiêu; total penalty chỉ chọn nghiệm đại diện và đo tiến bộ sau khả thi.

Sectioning: greedy trước, capacity-plan fallback khi cần

- Greedy xử lý sinh viên nhiều course trước; beam 12 chọn bundle class. Nếu còn missing requests, giải lại toàn bộ người đăng ký course bị lỗi.
- Sinh bundle hợp lệ → CP-SAT tìm số người dùng mỗi bundle → ghép lại sinh viên theo xung đột với course khác → tính lại missing/conflicts.
- Giới hạn 4.096 bundles / 200.000 nodes; CP một worker, seed cố định, deterministic budget 0,5. Structural capacity plan được cache theo instance/course.
- Không tìm được plan trong tập bundle đã sinh không chứng minh XML vô nghiệm. Fallback ưu tiên sức chứa, chưa bảo đảm student penalty tối ưu.

Truy vết: instance.py; scoring.py; sectioning.py. Bản đồ module chi tiết ở cuối báo cáo.

02 / VÒNG LẶP TIẾN HÓA

Từ construction đến nghiệm đại diện

1. Construction có xét quan hệ ràng buộc

- Ưu tiên domain nhỏ, mức liên kết và seeded tie-break; xét required pairs với class đã đặt, partial global costs, room usage và unavailable.
- Forward checking tối đa 4 hàng xóm; lan truyền pair-compatible domains. SameRoom/SameStart tận dụng giao domain nếu có.
- Mỗi class lấy mẫu tối đa 32 time / 16 room, giữ tối đa 64 placements đa dạng. Domain bị cắt chỉ là giới hạn tìm kiếm, không phải bằng chứng vô nghiệm.

2. Initial repair và variation

- Initial repair dùng single moves rồi bounded two-class moves. Bỏ lượt RoomRepair tự động ngay sau sampling để tránh repair hai lần.
- Crossover giữ cặp time/room theo class; mutation thay một phần nhỏ class. Offspring repair sửa tại chỗ quanh vi phạm, không dựng lại toàn timetable.
- Routine joint search xét exhaustive khi product domain ≤ 96; domain lớn dùng lấy mẫu. Deadline được kiểm bên trong vòng lặp.

3. Full scoring và NSGA-III survival

- Full scorer đánh giá placement, required/soft distributions và decoded student assignment. NSGA-III dùng feasibility handling và bốn objective để chọn quần thể tiếp.
- Tournament/selection dùng RNG của solver; seed được ghi cùng config. Không thay survival bằng xếp hạng một scalar total.

4. Incumbent dùng chung trước survival

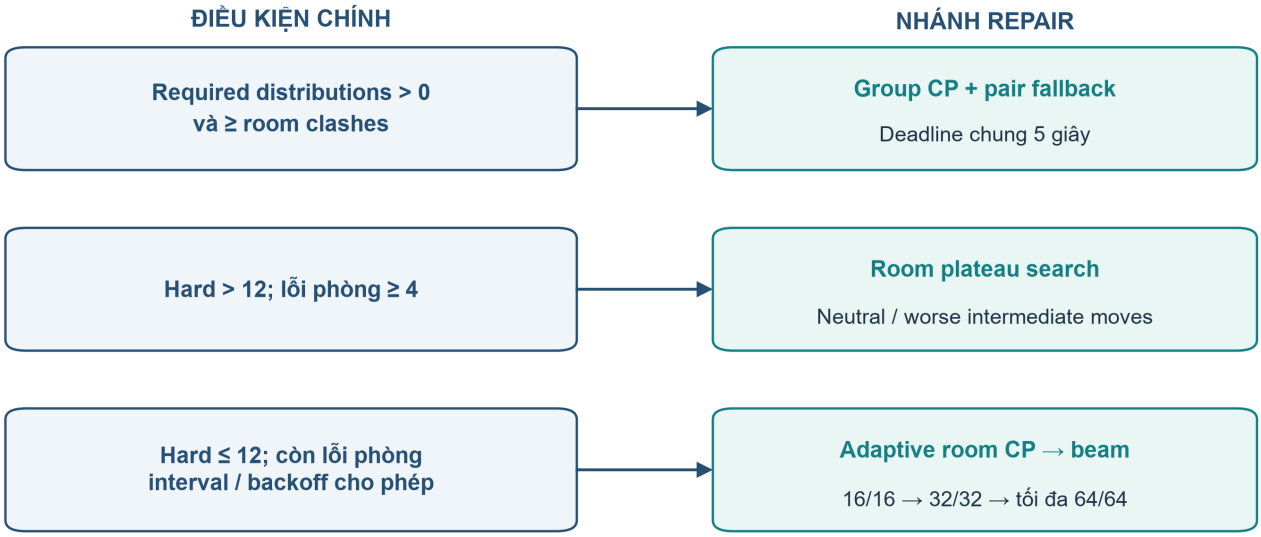
- Mọi candidate đi qua PlacementProblem.score_state đều có thể cập nhật incumbent theo thứ tự (hard, total), kể cả candidate do repair hoặc operator sinh ra.
- Candidate tốt bị survival loại vẫn còn trong incumbent. Monitor, controller feedback, progress và output cùng tham chiếu archive này.
- Archive hiện giữ một nghiệm đại diện; chưa có external Pareto archive đầy đủ.

Hard 0 không phải điều kiện dừng bắt buộc

Mặc định solver tiếp tục cải thiện soft sau khi khả thi. Chỉ bật dừng sớm bằng --stop-on-feasible; nếu không, dừng khi chạm một trong các cap đã cấu hình.

03 / REPAIR DÙNG CHUNG

Các nhánh sửa được gọi như thế nào?



Các guard được xét theo thứ tự; không phải ba nhánh loại trừ hoàn toàn.

Routine repair luôn thuộc construction / variation; full scorer xác nhận kết quả.

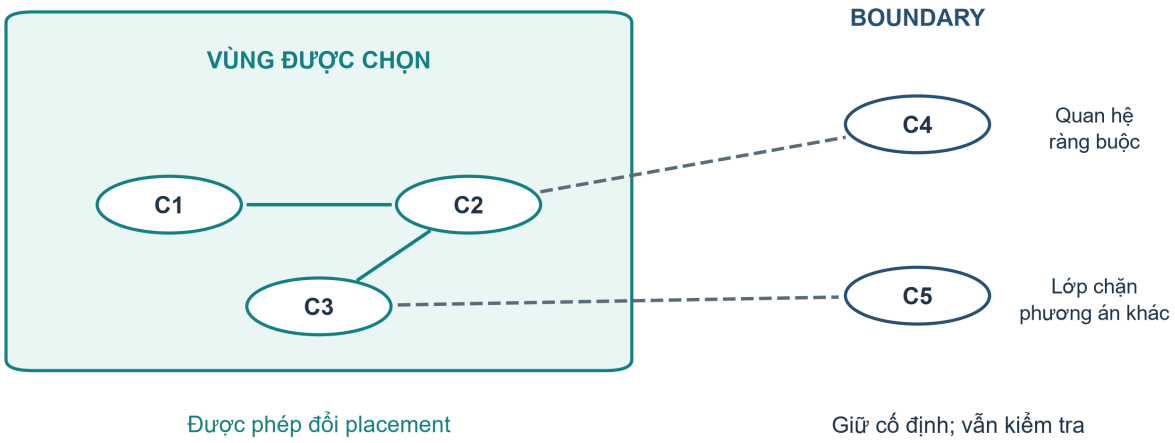
Hình 2. Các gate chính trong monitor, theo thứ tự xét. Sơ đồ rút gọn không liệt kê mọi guard: remaining budget, duplicate source, deadline và cooldown vẫn được kiểm tra.

Nhánh	Budget và acceptance
Distribution group CP	Mốc 5 generations; deadline chung 5 giây. Còn thời gian mới thử tối đa 3 rounds / 8 pairs mỗi round. Chỉ cần giảm tổng hard.
Room plateau	Tối đa 5 giây / 16 full checks; cho phép neutral/worse intermediate states. Chỉ trả cải thiện cuối.
Adaptive room CP	16/16 → preset 32/32 → tối đa 64/64 class/options trong cùng deadline/check budget. Mặc định 8 giây; yach dùng 15 giây theo profile.
Beam fallback / backoff	Fallback tối đa 2 giây / 128 full checks nếu còn budget. Cấu trúc lỗi thử thất bại được hoãn 5 → 10 → 20 generations.

- Room neighborhood mở sang lớp đang chặn phương án thay thế, kể cả lớp hiện không vi phạm; phần ngoài vùng được cố định.
- CP mã hóa room clash/unavailability, pairwise required constraints, MaxDays, MaxDayLoad, MaxBreaks và MaxBlock, kể cả fixed-boundary interactions.
- Room branch phải giảm tổng hard và lỗi phòng. Full scorer kiểm lại toàn timetable/sectioning; CP không thay thế scorer hay giải toàn instance.
- UNKNOWN là chưa giải xong trong model/budget. INFEASIBLE của vùng đã cắt domain cũng không chứng minh toàn XML vô nghiệm.

04 / WHERE VÀ WHEN

Từ search state đến vùng can thiệp



Hình 3. Minh họa cấu trúc region/boundary, không phải timetable lấy từ dataset. Các node trong vùng được phép đổi; quan hệ với phần ngoài vẫn phải được bảo toàn qua validation và full scoring.

WHERE: StatePacket được solver dựng, không do LLM tự đọc toàn lịch

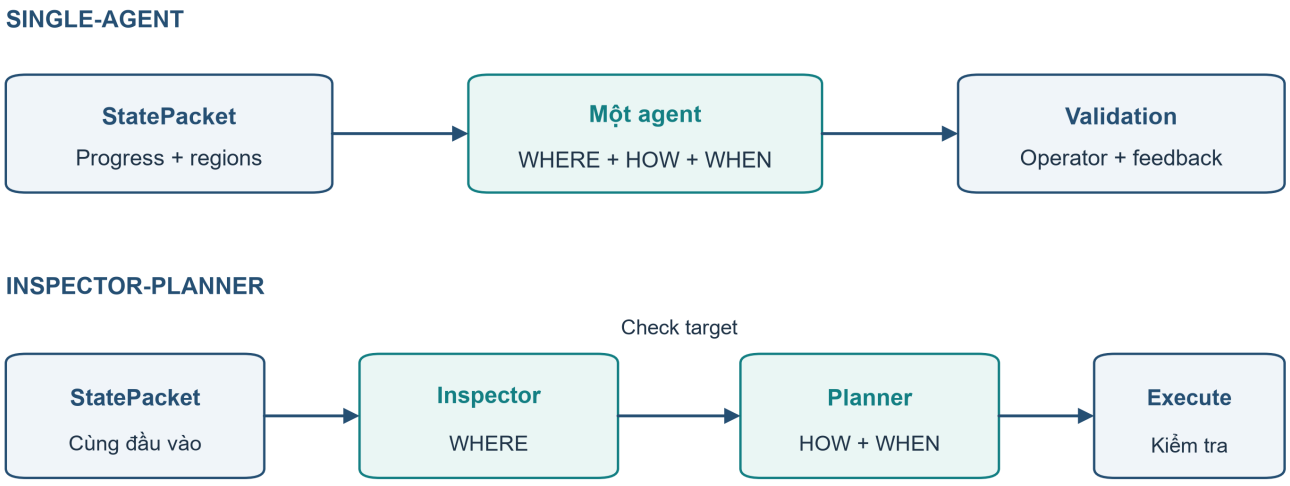
- Chọn tối đa hai representatives: nghiệm tốt nhất theo (hard,total) và một chromosome khác, ưu tiên trade-off objective/placement khi hard gần tương đương.
- Tạo tối đa 6 connected regions, mặc định 5 class/vùng. Pairwise violations quy về đúng endpoints; student conflicts lấy từ decoded assignment.
- Global-group attribution vẫn heuristic; missing course requests đánh dấu unlocalized, không gán cho một vùng tùy ý.
- Boundary chứa distribution relationships và alternative-room blockers. Packet hiển thị tối đa 32 boundary IDs kèm count/truncated; nội bộ giữ đầy đủ.
- Packet gồm progress/diversity, regions và allowed moves, 5 outcomes gần nhất, action limits. Incumbent metrics và population metrics được phân biệt.

WHEN: trigger theo sự kiện và checkpoint

- Warm-up generation 5; khoảng cách tối thiểu 3 generations. Checkpoint ban đầu 10; agent đặt interval tiếp theo trong [3,20].
- Review khi đến checkpoint, stagnation 5 generations hoặc placement diversity ≤ 0,20. Không gọi ở mọi generation.
- Low-diversity chỉ rearm khi diversity > 0,30; stagnation rearm sau tiến bộ. Progress theo hard trước khả thi, total sau khả thi.
- Tối đa 12 reviews; profile cấp single 12 calls, multi 24 calls; timeout 15 giây/call, output 300 tokens. Baseline không gọi API.

05 / HOW VÀ VALIDATION

Agent đề xuất; solver quyết định thực thi



Cùng solver, operators, acceptance và ngân sách; chỉ khác cách phân vai.

Hình 4. Single-agent và Inspector-Planner dùng cùng StatePacket, action space và safety rules. Ở multi-agent, target được xác minh trước Planner; thời gian và tokens của cả hai calls đều được ghi.

Action	Thực thi trong vùng được xác minh
move	Đổi time hoặc room của một class.
swap	Hoán đổi hai placements tương thích với domain của cả hai class.
destroy_repair	CP sửa đồng thời toàn vùng, tối đa 3 giây; còn evaluation budget mới fallback tuần tự tối đa 3 class.
no_op	Không đổi timetable; vẫn có next review interval hợp lệ.

- **Kiểm tra response:** action, IDs, target/boundary, operator, evaluation budget và next-review interval đều phải hợp lệ. Agent không chạy mã tự sinh.
- **Source chưa khả thi:** candidate phải giảm hard, không bắt lập tức về hard 0. **Source khả thi:** giữ hard 0 và giảm soft total.
- Áp operator trên bản sao, kiểm tra toàn instance rồi đưa candidate vào survival. Mọi full-score candidate vẫn cập nhật incumbent trước đó.
- Response lỗi thành no_op có reason. Lỗi cấu hình API ngừng review tiếp theo của instance; không âm thầm coi đó là quyết định hợp lệ.
- Local CP soft objective ưu tiên weighted time/room. Student/soft-distribution penalties được full scorer kiểm, chưa tối ưu trực tiếp đầy đủ trong CP.

06 / NGÂN SÁCH VÀ CẤU HÌNH

Cấu hình hiện chọn không phải CLI defaults

Setting	Main.py mặc định	Profile hiện chọn
Population / H	20 / 3	12 / 2 cho nbi, yach; 20 / 3 cho bốn instance khác
Mutation rate / fraction	0,20 / 0,005	0,35 / 0,01
Crossover	0,70	0,70
Initial / offspring repair	12 steps, 3 pair trials / 8 steps, 2 pair trials	Giữ nguyên
Inner directions	Tắt	Tắt; vẫn có tùy chọn CLI
Seed mặc định	42	Wrapper dùng 17 nếu không truyền seed

Caps theo instance trong profile

Instance	Gen cap	Eval cap	Wall cap (s)
muni-fi-spr16	200	1.200	840
muni-fsps-spr17	200	1.200	630
yach-fal17	200	1.200	1050
nbi-spr18	200	1.200	750
mary-fal18	200	1.100	810
bet-spr18	230	2.300	2880

- **Chỉ wrapper tự đọc profile:** dùng run_pilot.py. Gọi main.py trực tiếp dùng CLI defaults, trừ khi truyền đủ flags.
- Wrapper merge solver/repair overrides một lần và áp như nhau cho cả ba pipeline. So cùng instance phải dùng cùng profile và seed.
- Caps là hàng rào; run dừng khi một điều kiện chạm trước. Thêm repair evaluations khiến cùng evaluation cap không nhất thiết đạt cùng generation thực tế.
- Wall cap có hệ số an toàn, dự phòng generation dài và tối đa 360 giây API latency; không phải dự báo runtime baseline hay đo API thật.
- Preset chỉ được chọn trong miền đã thử. muni-fi/yach còn tiến triển ở evaluation cap; bet có endpoint bị time cap; recalibration_required vẫn true.

07 / CHẠY, BUDGET ACCOUNTING VÀ OUTPUT

Đọc đúng một lần chạy

Logical evaluations và thời gian

- ScoreEngine là LRU cache tối đa 16 chromosome/instance, giữ score và assignments. Cache hit vẫn tiêu logical evaluation; physical calls/cache hits ghi riêng.
- Search count cộng objective checks của Pymoo, routine/monitor repair, region scoring, operators và rechecks. Đối chiếu với score_engine.logical_requests.
- Local deltas/CP nodes chỉ tính runtime, không giả thành full evaluation. Hai final verification calls được liệt kê riêng ngoài search budget.
- Reserve allowance trước repair/controller; batch offspring cuối thu nhỏ để giữ evaluation cap. Deadline hợp tác: full score/initialization đã bắt đầu vẫn có thể hoàn tất sau wall cap.

Lệnh từ thư mục Experiment, trong venv đã cài requirements

```
python run_pilot.py --pipeline baseline --seed 17 --dry-run
python run_pilot.py --pipeline baseline --seed 17
python run_pilot.py --pipeline baseline --instance muni-fsps-spr17 --seed 42
```

Đổi pipeline thành single/multi và cấu hình model/API key để dùng LLM. Các lệnh agent có thể phát sinh chi phí API. Báo cáo này không thực thi các lệnh trên.

Artifact	Dùng để đọc gì?
run_config.json / summary.json	Config, định danh run và tóm tắt các instance.
result.json	Score/components, evaluation breakdown, timing, sectioning và repair events.
choices.npz	Chromosome đã lưu để tái chấm hoặc probe riêng.
anytime_trace.jsonl	Tiến trình incumbent/population theo generation và budget.
controller_trace.jsonl	Trigger, quyết định, validation, outcome và chi phí agent.
solution.xml	Chỉ xuất khi nghiệm cuối hard 0 và sectioning qua kiểm tra nội bộ.

- Run ID gắn settings, XML/source hashes và dependency versions. Cùng định danh chỉ thay output sau khi run mới hoàn tất qua staging.
- Không có solution.xml không có nghĩa run thất bại kỹ thuật: xem hard, components, stop reason và trace trước.

08 / TRẠNG THÁI VÀ TRUY VẾT

Đã có gì; giới hạn nào vẫn còn?

Trạng thái đã ghi nhận, không phải cam kết cho mọi instance

- Đã có shared incumbent, group CP, forward-checking construction, adaptive/global CP, cache/guards, regional joint operator và capacity-plan sectioning fallback.
- Trong các cặp seed xác nhận, muni-fi, muni-fsps, mary và nbi đạt khả thi nội bộ. yach còn room clashes; bet còn lỗi phòng và required distributions.
- Missing student requests không gây hard còn lại ở các endpoint này. Room/group CP gates vẫn có thể bỏ qua lỗi hỗn hợp; ép CP riêng chưa giải được ca bet trong budget đã thử.
- Chưa chạy validator độc lập hoặc đánh giá LLM thật trong đợt calibration. Không coi UNKNOWN là vô nghiệm hay endpoint bị time cap là hội tụ.

Có thể bắt đầu test controller trước khi mọi instance đạt hard 0

Với instance chưa khả thi, theo dõi hard/components và tiến trình giảm hard. Khi đã khả thi, theo dõi thời điểm đạt hard 0 và soft quality. Giữ cùng solver/profile/seed; không trộn soft scores của nghiệm khả thi và chưa khả thi thành một kết luận.

Nhóm chức năng	Module chính trong NSGA-III/itc2019
Dữ liệu / construction	instance.py; construction.py
Scoring / student assignment	scoring.py; sectioning.py
NSGA-III / archive / routine repair	nsga3.py
Monitor / CP / room search	controller.py; neighborhood.py; repair_policy.py; room_search.py
StatePacket / actions / API	regions.py; operators.py; llm.py
Run / kiểm tra tương thích	run_pilot.py; compare_pipelines.py (ở thư mục Experiment)

Giới hạn còn lại: search có sampled domains và bounded neighborhoods; capacity fallback có thể bỏ lỡ bundles; chưa có external Pareto archive; local CP chưa tối ưu trực tiếp mọi soft objective. Không khẳng định solver tối ưu toàn cục hoặc giải được toàn ITC 2019.